Tecnología para la Gestión II

Clase 2 — Proyectos, carga e inspección de datos

Francisco Villarroel Riquelme | 2026-08-11

Administración Pública · Escuela de Gobierno · Facultad de Economía y Gobierno · ADPU B012

¿Qué vimos la clase pasada?

R ≠ RStudio R es el motor; RStudio, el tablero desde donde lo manejamos.

Los 4 paneles Script, Environment, Consola y Output (Files/Plots/Packages/Help).

Cargar datos `read.csv`, `read_excel`, `readRDS`... los datos entran como un objeto.

Hoy le damos **orden y método** a todo eso: trabajar con **proyectos**, cargar bien las bases y **mirar los datos antes de tocarlos**.

Agenda de hoy

Bloque 1 — Orden y carga

- Qué es un proyecto y por qué usarlo
- Estructura de carpetas (`data`, `scripts`, `results`)
- Instalar librerías
- Activar librerías
- Cargar bases de datos

Bloque 2 — Mirar los datos

- Inspección de datos: `names()`, `glimpse()`, `class()`, `view()`
- Tipos de variables en R *(si alcanzamos)*
- **Control breve** al final

Bloque 1

Proyectos, orden y carga de datos

¿Qué es un proyecto en RStudio?

Un **proyecto** (`.Rproj`) es una **carpeta de trabajo autocontenida**: agrupa en un solo lugar tus datos, tus scripts y tus resultados, y le dice a R que ese es el punto de partida para trabajar.

Al abrir el `.Rproj`, el **directorio de trabajo** queda fijado automáticamente en esa carpeta. No tienes que andar escribiendo `setwd("C:/Users/...")` nunca más.

¿Por qué usar proyectos? → Replicabilidad

Sin proyecto ❌

```
read.csv("C:/Users/Francisco/
   Desktop/cosas/base_87.csv")
```

Esa ruta **solo existe en tu computador**. En otra máquina, el código se cae.

Con proyecto ✅

```
read.csv("data/base_87.csv")
```

Ruta **relativa** a la carpeta del proyecto. Funciona en **cualquier** computador que tenga la carpeta.

Replicabilidad = que **otra persona (o tu yo del futuro)** pueda abrir la carpeta y correr todo sin cambiar nada. Es la base del trabajo serio con datos.

Otras ventajas del proyecto

Orden Todo lo de un análisis vive junto: nada de archivos perdidos en el Escritorio.

Portabilidad Comprimes la carpeta, la envías, y funciona igual en otro equipo.

Historial limpio Cada proyecto recuerda sus scripts abiertos y su entorno.

Cómo crear un proyecto

1. En RStudio: **File** → **New Project…**
2. Elige **New Directory** → **New Project**
3. Ponle un **nombre** (ej. `clase2`) y elige **dónde** guardarlo
4. Click en **Create Project** → RStudio se reinicia dentro del proyecto

> Sabrás que estás dentro del proyecto porque su nombre aparece **arriba a la derecha** de RStudio, y la pestaña *Files* muestra la carpeta del proyecto.

Estructura de carpetas recomendada

Dentro del proyecto, separa por **función** lo que haces:

```
clase2/
├── clase2.Rproj      # el proyecto (no se toca)
├── data/             # bases de datos crudas (.csv, .xlsx, .dta...)
├── scripts/          # tu código .R
└── results/          # gráficos, tablas e informes que generas
```

data/ entra, **results/** sale. Los datos crudos **no se modifican**: lo que transformas queda en el script y lo que produces va a `results/`.

Con esta estructura, tus rutas son cortas y claras: `read.csv("data/base_87.csv")` y `ggsave("results/grafico1.png")`.

Librerías (paquetes)

Instalar ≠ activar

1) Instalar librerías

Un **paquete** es una colección de funciones que amplía lo que R puede hacer.

```
install.packages("readxl", dependencies = TRUE)
install.packages("tidyverse", dependencies = TRUE)
install.packages("haven", dependencies = TRUE)
install.packages("car", dependencies = TRUE)
```

> `install.packages("nombre")` = *"instala el paquete"*. El argumento `dependencies = TRUE` le pide que instale también **las librerías adicionales** que ese paquete necesita para funcionar.

Se instala UNA vez por computador. Quedan disponibles en la pestaña *Packages* (abajo a la derecha).

2) Activar (llamar) librerías

Instalar **no es suficiente**: hay que **llamar** el paquete en cada script.

```
library(readxl)
library(tidyverse)
library(haven)
library(car)
```

Regla de oro: **`install.packages()` una vez** en el computador; **`library()` cada vez** que abres un script nuevo. Si abres R y una función "no existe", casi siempre es que te faltó el `library()`.

3) Cargar bases de datos

Ya con las librerías activas, cargamos la base como un **objeto**:

```
# CSV separado por comas
cep <- read.csv("data/base_87.csv")

# ¿Cargó todo en una sola columna? Usa punto y coma:
cep <- read.csv2("data/base_87.csv")
```

> Aparece el objeto `cep` en el panel **Environment** con sus dimensiones: por ejemplo, 2915 observaciones (filas) y 2019 variables (columnas).

Cargar otros formatos

RStudio abre casi cualquier formato; a veces con un paquete específico:

```
# Excel
cep <- readxl::read_excel("data/base_87.xlsx")

# STATA
cep <- haven::read_dta("data/base_87.dta")

# SPSS
cep <- haven::read_spss("data/base_87.sav")

# Formato propio de R
cep <- readRDS("data/base_87.rds")
```

> El `::` indica **de qué paquete** viene la función. `readxl::read_excel` = "la función `read_excel`, del paquete `readxl`". Sirve para diferenciar funciones que se llaman igual en paquetes distintos.

Bloque 2

Inspección de datos

Antes de tocar, hay que mirar

Con los datos cargados, necesitamos conocer su **naturaleza**: el tamaño de la base, el tipo de dato de cada columna, los nombres de las variables y sus categorías.

Cuatro funciones para empezar:

`names()`

los nombres de las variables

`glimpse()`

un vistazo general

`class()`

el tipo de una variable

Y `view()` para abrir la base completa en una pestaña. Basado en tu material de RPubs (sección 2.2).

`names()` — los nombres de las columnas

Entrega el nombre de **cada variable** del dataframe:

```
names(cep)
```

> En la CEP los nombres vienen **codificados** (`interes_pol_1_b`, `region`...). Para entenderlos hay que revisar el **libro de códigos** de la encuesta. Esto pasa con casi todas las bases públicas: conocer bien la base es un trabajo indispensable.

`glimpse()` — el vistazo más completo

```
glimpse(cep)
```

Es la que **más información** entrega de una sola vez:

- Dimensiones de la base
- Nombres de las variables
- **Tipo** de cada variable
- Una muestra de sus valores

Abreviaturas del tipo:

- `int` = integer (enteros)
- `dbl` = double (con decimales)
- `chr` = character (texto)

`glimpse()` viene con el `tidyverse` (paquete `dplyr`), así que recuerda tener activo `library(tidyverse)`.

`class()` y el operador `$`

Para saber el tipo de **una** variable específica:

```
class(cep$interes_pol_1_b)    # [1] "integer"
class(cep$region)             # [1] "character"
```

> El signo **`$`** accede a una variable dentro de un dataframe: `cep$region` = "la variable `region`, dentro de `cep`". Acá vemos que una es numérica (`integer`) y la otra es texto (`character`).

`view()` — ver la base completa

Abre la base como una **pestaña** aparte (arriba a la izquierda), tipo planilla:

```
view(cep)
```

> Útil para explorar visualmente, pero **no reemplaza** a `glimpse()` / `class()`: para trabajar y documentar, siempre preferimos las funciones que dejan registro en el script.

Flujo recomendado al abrir cualquier base: **`names()` → `glimpse()` → `class()` de las variables clave → `view()`** para confirmar.

Tipos de variables en R

Los tipos que más usaremos

logical Booleanos: solo `TRUE` / `FALSE`. Útiles para condiciones y filtros.

```
boul <- c(TRUE, FALSE, TRUE)
class(boul)  # "logical"
```

integer Enteros, sin decimales. Llevan el sufijo `L`.

```
x <- 186L
class(x)  # "integer"
```

double / numeric Números (con o sin decimales). Permiten operaciones matemáticas.

```
altura <- c(1.67, 1.98, 1.50)
class(altura)  # "numeric"
```

character Texto entre comillas. Categorías que se pueden contar.

```
genero <- c("f", "m", "otro")
class(genero)  # "character"
```

string En rigor es `character`, pero para **frases** y texto complejo.

```
w <- "hola, ¿qué tal?"
class(w)  # "character"
```

factor Categoría con **niveles** ordenables. *(siguiente diapo)*

El caso especial: `factor`

Un **factor** es una variable categórica que guarda un **orden** (niveles). El problema: el texto se ordena **solo alfabéticamente**.

```
clase <- c("clase baja", "clase alta", "clase media")
table(clase)   # aparece en orden alfabético: alta, baja, media  ✗
```

Lo convertimos a factor y le damos el **orden correcto**:

```
clase <- as.factor(clase)
levels(clase) <- c("clase baja", "clase media", "clase alta")
table(clase)   # ahora: baja, media, alta  ✓
```

> Ojo: para transformar un objeto **hay que reasignarlo**. No basta `as.factor(clase)`, tiene que ser `clase <- as.factor(clase)`. Los factores nos servirán mucho para **gráficos** y **regresiones**.

Control breve

5 preguntas + 2 ejercicios

Parte teórica *(marca la alternativa correcta)*

1. ¿Cuál es la principal ventaja de trabajar con proyectos en RStudio? a) Hace que R corra más rápido b) Permite la replicabilidad usando rutas relativas c) Instala los paquetes automáticamente d) Convierte los .csv en .xlsx

2. ¿Cuál es la sintaxis correcta para *instalar* un paquete? a) `library("readxl")` b) `install("readxl")` c) `install.packages("readxl", dependencies = TRUE)` d) `load(readxl)`

3. Después de instalar un paquete, ¿qué necesitas para usarlo en un script? a) Nada, queda activo para siempre b) Llamarlo con `library()` en cada script c) Reiniciar el computador d) Volver a instalarlo cada vez

Parte teórica *(continuación)*

4. ¿Qué función entrega las dimensiones de la base, los nombres de las variables, su tipo y una muestra de los datos? a) `names()` b) `class()` c) `glimpse()` d) `view()`

5. Una variable categórica que guarda un *orden* mediante niveles (levels) es de tipo: a) `character` b) `logical` c) `numeric` d) `factor`

Tiempo sugerido: 4–5 minutos para la parte teórica.

Parte práctica

Ejercicio 1 — Proyecto y carga Crea un proyecto nuevo llamado `clase2` y, dentro, las carpetas `data`, `scripts` y `results`. Deja un archivo `.csv` en `data/`. Desde un script en `scripts/`, cárgalo en un objeto llamado `datos` usando una **ruta relativa**.

Ejercicio 2 — Inspección y tipos Con `datos` cargado, escribe las líneas para: (a) ver los **nombres** de las variables, (b) obtener un **vistazo general** con dimensiones y tipos, y (c) consultar la **clase** de una variable específica usando `$`. Luego crea el vector `clase <- c("clase baja","clase media","clase alta")` y conviértelo en **factor** con los niveles ordenados.

La próxima clase: manipulación de datos

`select()`, `filter()`, operadores lógicos y *pipes* (`%>%`)

Dudas: fvillarroelr1@docente.uss.cl · Tecnología para la Gestión II · ADPU B012

Pauta del control solo docente

Teóricas

1. **b)** replicabilidad con rutas relativas
2. **c)** `install.packages("readxl", dependencies = TRUE)`
3. **b)** llamarlo con `library()` en cada script
4. **c)** `glimpse()`
5. **d)** `factor`

Prácticas (idea de solución)

Ej. 1:

```
# tras crear el proyecto y las carpetas
datos <- read.csv("data/mi_base.csv")
```

Ej. 2:

```
names(datos)
glimpse(datos)
class(datos$mi_variable)

clase <- c("clase baja","clase media","clase alta")
clase <- as.factor(clase)
levels(clase) <- c("clase baja","clase media","clase alta")
```

Elimina esta diapositiva antes de proyectar si no quieres mostrar la pauta.